GUIA DO PROFESSOR

Sensor de distância: do eco do som ao alarme que avisa pelo celular

Plano de 6 aulas, física e fundamentação, gabarito dos desafios, solução de problemas e avaliação

---

Placas: Arduino Uno → ESP32 • **Componentes:** HC-SR04, buzzer MH-FMD, LED RGB

Projetos: gráfico ao vivo em Python • alarme de ré • monitor web local • alerta por WhatsApp e e-mail

Público: Ensino Médio / Técnico • **Carga horária:** 6 aulas de 100 min

Espaço Cria — 2026

Sumário

1. Visão geral
2. Materiais
3. Preparação
4. Plano das aulas
5. Fundamentação para o professor
6. Erros comuns e como resolver
7. Gabarito e comentários dos desafios
8. Avaliação
9. Diferenciação
10. Segurança
11. Referências

Anexo A — Esquemas

Anexo B — Sketch final de referência

1. Visão geral

O HC-SR04 é o sensor ideal para uma sequência longa: é barato, robusto, tem **física visível** (som, eco, velocidade, temperatura), produz um dado numérico que precisa de **tratamento** (conversão, filtro), e permite chegar a **sistemas completos** — aquisição de dados no PC, atuadores, rede local e internet. A sequência foi desenhada em espiral: cada aula reusa o código da anterior e acrescenta uma camada (medir → visualizar → sinalizar → integrar → conectar → notificar), com ênfase em **explicar o porquê** — a regra de três da distância, a mediana, o divisor de tensão, o protocolo SMTP visto em texto.

Objetivos de aprendizagem

- Explicar como o tempo de voo do som vira distância (proporcionalidade) e quais fatores afetam a medida (temperatura, ângulo, material).
- Usar `pulseIn`, temporização em µs e a porta serial; formatar dados (CSV) para outros programas.
- Aplicar filtros (mediana, média móvel) e justificar a escolha.
- Ler a serial em Python, converter texto em números e produzir um gráfico ao vivo.
- Explicar buzzer ativo × passivo, frequência e nota; representar uma melodia como dados (vetores).
- Projetar um sistema reativo sem `delay()` (multitarefa por `millis()`), com regras em tabelas.
- Migrar para o ESP32 respeitando níveis lógicos (divisor de tensão) e servir dados em JSON para uma página.
- Descrever o caminho de um alerta até a internet (Wi-Fi, HTTPS, API, SMTP) e os cuidados com credenciais e cooldown.

Competências (BNCC / Itinerários)

- **Física (EM13CNT):** ondas sonoras, velocidade de propagação, reflexão, frequência e altura; tensão, resistência e divisor.
- **Matemática (EM13MAT):** proporcionalidade, unidades (µs ↔ cm), estatística descritiva (média, mediana, desvio), gráficos de função do tempo.
- **Pensamento computacional:** decomposição, abstração (classes/rotas), máquinas de estado, tratamento de erros, protocolos.
- **Cultura digital / mundo do trabalho:** aquisição de dados, IoT, automação, segurança de credenciais.

Pré-requisitos

Ter feito a sequência de LEDs (ou equivalente): `pinMode/digitalWrite/digitalRead`, `if/for`, funções, `millis()` e `analogWrite`. Noções de Python (variáveis, `print`) ajudam na Aula 2, mas o guia explica o necessário.

2. Materiais

Por grupo (2–3 alunos)

Item	Qtd	Custo aprox. (R$)	Observação
Arduino Uno + cabo USB	1	40–90	Aulas 1–4
ESP32 Dev Module + cabo micro-USB de dados	1	35–60	Aulas 5–6
Sensor HC-SR04	1 (+1 reserva/turma)	8–15	Versões com "+"/RCWL-1601 funcionam em 3,3 V, mas o guia assume o clássico
Buzzer passivo MH-FMD (ou KY-006)	1	3–6	Confirmar que é **passivo**
LED RGB cátodo comum + 3 × 220 Ω	1	2	
Resistores 1 kΩ e 2 kΩ (ou 2,2 kΩ)	1 + 1	0,20	Divisor do ECHO (Aula 5)
Protoboard 830 + jumpers	1	25–40	
Régua/trena	1	—	Aula 1

Para a turma

- PCs com Arduino IDE 2 + core ESP32, **Python 3** com `pyserial` e `matplotlib` (Aula 2). No laboratório do projeto, use `uv run` na pasta `python-dados`.
- Celulares dos alunos (Aula 5) e **uma rede Wi-Fi com internet** cujo nome/senha possam ir no código (Aula 6) — idealmente um roteador/hotspot só para a aula.
- Um número de WhatsApp e uma conta Gmail **dedicados ao laboratório** (não pessoais) para a Aula 6.
- Projetor: figuras em `aula-distancia/esquemas/saida/*.png`.

3. Preparação

1. IDE com o core **esp32** (≥ 3.0). O sketch 05 **não** usa biblioteca de e-mail (fala SMTP direto), então não há nada extra para instalar. (Se preferir a biblioteca *ESP Mail Client*, mude a partição para *Huge APP*; ela não cabe na padrão.)
2. Python: `pip install pyserial matplotlib` (ou `uv add`). Teste sem hardware: `python plot_distancia.py --simular`.
3. Todos os sketches foram **compilados**: Uno 01, 01b, 02, 02b, 03; ESP32 04, 05. Copie `aula-distancia/codigo` e `aula-distancia/python` para as máquinas.
4. **Aula 6 — configure antes:** ative o CallMeBot com o número do laboratório (contato +34 644 87 21 57, mensagem `I allow callmebot to send me messages`) e crie a **senha de app** do Gmail. Guarde APIKEY e senha num arquivo fora do repositório; distribua na hora e apague depois.
5. Teste o sketch 05 com as suas credenciais antes da aula. Se a rede da escola bloquear SMTP (porta 465) ou HTTPS externo — comum em redes institucionais — use um hotspot de celular.
6. Verifique cada buzzer: ligado direto em 5 V, se apitar é **ativo** (não serve para melodias).

7. Os esquemas foram feitos no Fritzing: `aula-distancia/esquemas/saida/*.fzz` (editáveis; `python esquemas.py` regenera).

4. Plano das aulas

Aula 1 — Medindo distância com som

Tempo	Atividade	Professor	Alunos	Evidência
10'	Problema	“Como um morcego ou um submarino sabem a distância?” Demonstra o alarme final	Hipóteses	Quadro
20'	Física do eco	Velocidade do som; ida e volta; deduz a fórmula com a turma (regra de três no quadro); temperatura	Calculam o tempo esperado para 20 cm (1166 µs)	Cálculo no caderno
15'	Montagem + sketch 01	Confere VCC/GND; Monitor Serial 9600	Medem e comparam com a régua	**Ficha de medições** (anexo)
15'	Experimente	Travesseiro, superfície inclinada, delay curto	Observam e explicam	Explicações orais
15'	Porta serial e Plotter	Mostra o Plotter Serial; explica baud e “cano de texto”	Abrem o Plotter	—
15'	Sketch 01b (mediana, CSV)	Demonstra leitura louca no 01 e o efeito da mediana; por que não média	Comparam as curvas	Explicam mediana × média
10'	Desafios 1.x + fechamento	1.2 e 1.6 para todos; 1.4 para avançados. Ticket: “por que dividir por 2?”	Programam	Ticket

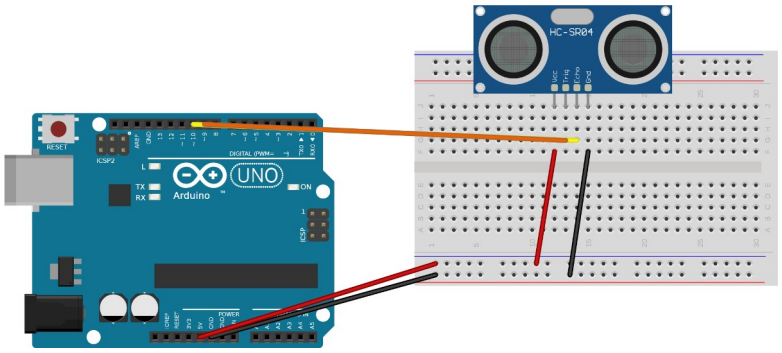


Figura 1 — Aula 1 — montagem básica.

Aula 2 — Python lendo a serial e plotando ao vivo

Tempo	Atividade	Professor	Alunos	Evidência
10'	Ideia	“O Monitor Serial é só um programa que lê a porta. O Python também pode.” Regra: uma porta, um programa	—	—
15'	Ambiente	Confere Python e bibliotecas; `ler_serial.py` sem argumentos lista as portas	Instalam/testam	Porta listada

Tempo	Atividade	Professor	Alunos	Evidência
20'	`ler_serial.py`	Explica bytes × texto, `strip`, `split`, `repr`; provoca erros (baud errado, monitor aberto)	Rodam e leem linhas	Terminal com dados
25'	`plot_distancia.py`	Explica as duas classes (abstração), `deque`, `FuncAnimation`, NaN; roda primeiro `--simular`	Rodam com o Arduino	Gráfico ao vivo
15'	CSV + planilha	Gravar 1 min e abrir no Calc; média/mín/máx	Analisam	Planilha
10'	Desafios 2.x	2.1 (velocidade) para todos; 2.4 introduz comunicação bidirecional	Programam	Script
5'	Fechamento	Ticket: "por que `time.sleep(2)` depois de abrir a porta?"	Respondem	Ticket

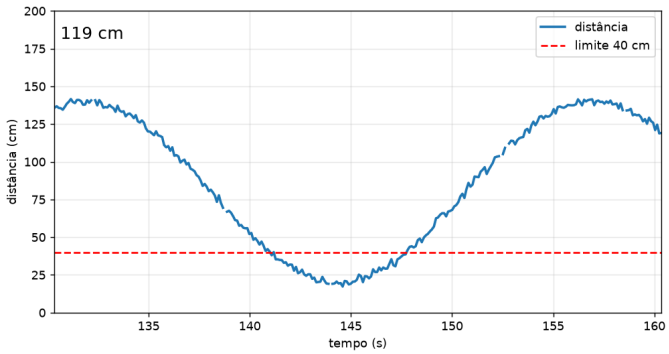


Figura 2 — Aula 2 — resultado esperado do `plot_distancia.py` (modo simulado).

Aula 3 — Buzzer e melodias

Tempo	Atividade	Professor	Alunos	Evidência
15'	Som e frequência	App de gerador de tom/afinador no celular: 440 Hz; oitava = dobro. Piezo: mostra o disco	Ouvem, comparam	—
10'	Ativo × passivo	Liga cada tipo direto em 5 V; explica o oscilador interno	Classificam os buzzers da caixa	Buzzers etiquetados
15'	Sketch 02	`tone` com e sem duração; sirene como `for` na frequência	Montam e testam	Sirene funcionando
25'	Sketch 02b	Vetor nota/duração; BPM → ms; nota pontuada; os 10 % de silêncio	Gravam; mudam o TEMPO	Explicam o vetor
15'	Repositório de músicas	Abre github.com/robsoncouto/arduino-songs; mostra como copiar uma música e trocar o pino	Escolhem e tocam uma música	Música tocando

Tempo	Atividade	Professor	Alunos	Evidência
15'	Desafios 3.x	3.5 (melodia sem travar) é a ponte para a Aula 4	Programam	Sketch
5'	Fechamento	Ticket: “o que muda no som se eu dobrar todas as frequências?”	Respondem	Ticket

Aula 4 — Mini projeto: alarme de ré

Tempo	Atividade	Professor	Alunos	Evidência
10'	Especificação	Vídeo de caminhão dando ré. “Escrevam a tabela: distância → cor → bipe”	Tabela no papel	Tabela
20'	Montagem	Sensor + RGB (pinos PWM 5, 6, 3) + buzzer; conferir GND comum	Montam	Checklist
20'	Sketch 03	Vetores paralelos como tabela; `faixaDe`; toggle por tempo; duas tarefas com `millis()`	Gravam; testam com a mão	Alarme funcionando
10'	Experimente	Mais faixas; frequência por faixa; mão no sensor (999)	Alteram a tabela	—
30'	Desafios 4.x	4.1 e 4.2 para todos (tempo de permanência e histerese são conceitos de automação real)	Programam	Sketch + explicação
10'	Fechamento	Ticket: “por que `INTERVALO = -1` significa silêncio? Que outro jeito daria?”	Respondem	Ticket

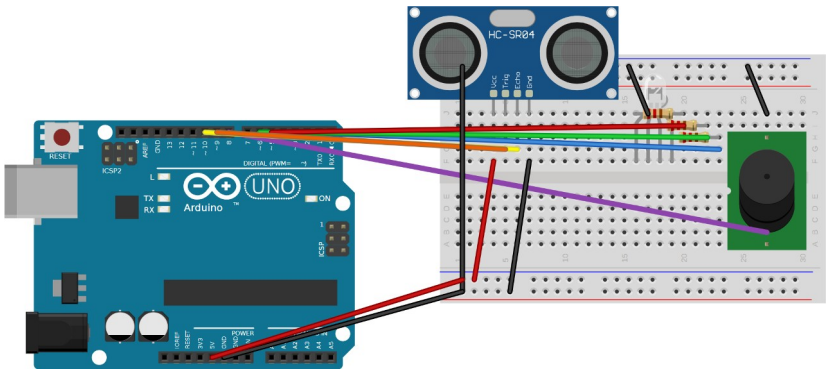


Figura 3 — Aula 4 — alarme de ré no Uno.

Aula 5 — ESP32 e monitor web

Tempo	Atividade	Professor	Alunos	Evidência
15'	5 V × 3,3 V	Mede com multímetro o ECHO em 5 V; deduz o divisor no quadro (regra de três de tensões)	Calculam 5 × 2/3	Cálculo
20'	Montagem ESP32	Confere o divisor (1 k no ECHO, 2	Montam	Multímetro: ~3,3 V no

Tempo	Atividade	Professor	Alunos	Evidência
		k no GND, GPIO no meio); sensor no VIN		GPIO 18
20'	Sketch 04	Rotas `/`, `/dados` (JSON), `/limite`; polling com `setInterval`; canvas	Gravam; conectam o celular	Página com gráfico
15'	Anatomia da página	Abre `/dados` direto no navegador (JSON cru); muda o intervalo do polling; muda cores	Personalizam	Página alterada
20'	Desafios 5.x	5.1 (histórico) e 5.2 para todos; 5.5 fecha o ciclo com o Python da Aula 2	Programam	Sketch/script
10'	Fechamento	Ticket: “o que é polling? Qual a desvantagem?”	Respondem	Ticket

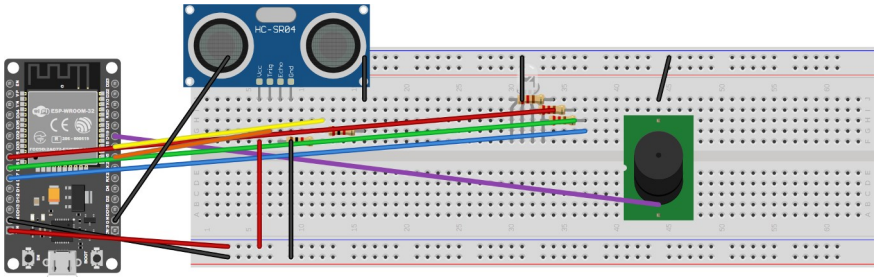


Figura 4 — Aulas 5–6 — montagem no ESP32 com divisor de tensão.

Aula 6 — Alerta pela internet

Tempo	Atividade	Professor	Alunos	Evidência
15'	Rede local × internet	Desenha no quadro: ESP32 → roteador → internet → servidor da API/Gmail → celular. O que é uma API	—	Desenho
15'	Credenciais	Distribui APIKEY/senha de app do laboratório; regras de sigilo; por que “senha de app”	Preenchem o sketch	—
20'	Sketch 05 — WhatsApp	URL da API; `urlEncode`; HTTPS/`setInsecure`; código 200	Testam; recebem a mensagem	Mensagem no celular
25'	Sketch 05 — SMTP	**Lê o diálogo no Serial com a turma** linha a linha: 220/EHLO/AUTH/base64/DATA/ `.`/QUIT. Base64 não é criptografia	Acompanham; recebem o e-mail	E-mail recebido
15'	Cooldown e bloqueio	Demonstra o cooldown; discute o que acontece sem ele (spam) e o LED azul travando o loop	Alteram COOLDOWN	—
10'	Fechamento e	Grupos demonstram o sistema	Apresentam	Rubrica

Tempo	Atividade	Professor	Alunos	Evidência
	apresentação	completo; rubrica		

5. Fundamentação para o professor

Física do HC-SR04

O módulo emite 8 ciclos a 40 kHz e mede o tempo até o receptor detectar o eco acima de um limiar. A largura do pulso ECHO é esse tempo (até ~38 ms se não houver eco). Com v = 343 m/s, d = v·t/2 → d(cm) = t(µs)/58,3. A velocidade varia com a temperatura (v ≈ 331,3 + 0,606·T m/s) e pouco com a umidade; a 0 °C o erro sistemático é −3,5 %. O cone de detecção é de ~15°; superfícies a mais de ~45° refletem o som para longe do receptor. Resolução típica 3 mm; precisão ±1 cm. O sensor precisa de ≥ 60 ms entre disparos para o eco residual decair — daí o `delay(100)`/`60 ms` nos sketches.

Regra de três: da medida física ao número

Vale destacar para os alunos que **toda** medida digital passa por uma conversão desse tipo: o sensor não "sabe" centímetros; ele entrega um tempo, e a fórmula (uma proporcionalidade) o transforma. O mesmo acontece com o ADC (`analogRead`: tensão → 0..1023), com o `map()` e com o divisor de tensão (resistências → tensão). Os quatro exemplos aparecem no curso e podem ser conectados explicitamente.

Filtros

A mediana rejeita **outliers** (leituras absurdas isoladas) sem atrasar muito; a média móvel suaviza ruído gaussiano mas é contaminada por outliers e atrasa a resposta (janela de 8 leituras a 60 ms = ~0,5 s). Uma combinação comum: mediana de 3–5 seguida de média móvel curta. É um bom momento para vocabulário estatístico com dados reais gerados pelos próprios alunos.

Serial e Python

A porta COM é um recurso exclusivo do sistema operacional: a IDE, o Python e o PuTTY competem por ela — origem do erro *acesso negado*. O auto-reset do Uno (DTR) reinicia a placa ao abrir a porta; por isso o `sleep(2)`. `readline()` bloqueia até `\n` ou até o `timeout`. Sobre `FuncAnimation`: a função de atualização deve ser rápida; por isso lemos no máximo 20 linhas por quadro. O simulador é a forma de a aula acontecer mesmo com metade dos Arduinos ocupados.

Buzzer e `tone()`

`tone()` usa o Timer2 do AVR e gera onda quadrada a 50 %; só um pino por vez. Conflita com `analogWrite` nos pinos 3 e 11 — por isso o RGB da Aula 4 usa 5, 6 e 3… cuidado: o pino 3 pode falhar com `tone` ativo; se acontecer, troque o azul para o pino 11 **ou** o buzzer para um pino sem PWM (8 está ok). No ESP32, `tone()` existe no core 3.x (usa LEDC). O buzzer passivo direto no pino consome < 20 mA; módulos MH-FMD têm transistor e aceitam 3,3 V.

Multitarefa por `millis()`

O sketch 03 tem duas "tarefas" com períodos diferentes; cada uma guarda o seu último instante. É a base de um escalonador cooperativo. O problema do sketch 05 (envio bloqueante de 2–7 s) motiva o desafio 6.3, que usa FreeRTOS (`xTaskCreatePinnedToCore`) — o ESP32 tem dois núcleos e um sistema operacional de tempo real embutido.

Níveis lógicos e divisor

Entradas do ESP32 toleram 3,6 V absolutos; ECHO a 5 V degrada o pino com o tempo. O divisor 1 k/2 k dá 3,33 V; 1 k/1,8 k dá 3,2 V. Corrente pelo divisor: 5/3000 ≈ 1,7 mA — desprezível. Alternativas: alimentar o HC-SR04 em 3,3 V (funciona mal, alcance reduzido) ou usar HC-SR04+/RCWL-1601 (3–5,5 V nativos).

HTTP, JSON e polling

A página usa `fetch` a cada 300 ms — simples e suficiente para 1–4 clientes. Alternativas: Server-Sent Events ou WebSocket (empurram dados; menos tráfego). JSON é montado "na mão" com `String`; para estruturas maiores, a biblioteca ArduinoJson. O `<canvas>` é desenhado com primitivas — sem Chart.js — para a página caber na flash e funcionar offline.

Internet: API, HTTPS, SMTP, segurança

- **CallMeBot** é uma API de terceiros, gratuita e só para o próprio número; pode mudar ou sair do ar. Alternativas equivalentes: Telegram Bot API (robusta, gratuita), ntfy.sh (push simples), IFTTT/Make (webhooks), Twilio (pago).
- **SMTP em texto**: o sketch 05 implementa o mínimo do RFC 5321 sobre TLS implícito (porta 465). Mostrar o diálogo no Serial é o momento mais "abre-olhos" do curso: e-mail é texto. `AUTH LOGIN` manda usuário e senha em **base64** — codificação, não cifra; o sigilo vem do TLS.
- `setInsecure()` desliga a verificação do certificado — o ESP32 fica sujeito a *man-in-the-middle*. Para produção, embutir o certificado raiz (`setCACert`). Discutir com a turma.
- **Senha de app** do Gmail: exige 2FA; pode ser revogada sem trocar a senha da conta. Nunca a senha principal. Não subir credenciais para GitHub (usar um `credenciais.h` no `.gitignore`).
- **Cooldown** é uma forma de *rate limiting* — obrigatório em qualquer sistema que notifica pessoas.
- Redes escolares frequentemente **bloqueiam** SMTP e portas não padrão; um hotspot de celular evita a frustração.

6. Erros comuns e como resolver

Sintoma	Causa provável	Solução
Sempre 0 / "fora de alcance"	TRIG e ECHO trocados; VCC em 3,3 V; objeto fora do cone	Conferir pinos; 5 V; apontar para parede plana
Leituras oscilando muito	Sem tempo entre medições; objeto macio/inclinado; fonte fraca	≥ 60 ms; mediana (01b); alvo rígido
Distância sempre ~1 cm ou fixa	Eco "curto-circuitado" pelo próprio suporte; TRIG preso em HIGH	Afastar do suporte; conferir `digitalWrite(TRIG, LOW)`
Python: PermissionError / acesso negado	Monitor Serial aberto ou outro script rodando	Fechar; `Ctrl+C` no outro terminal
Python: `could not convert string to float`	Linha de cabeçalho/incompleta	Ignorar linhas sem 2 partes numéricas (o script já faz)
Gráfico vazio	Baud errado; sketch 01 (não CSV) gravado	9600; gravar 01b

Sintoma	Causa provável	Solução
Buzzer só faz “tec”	Buzzer **ativo** com `tone`, ou passivo com `digitalWrite`	Trocar o tipo/código
Melodia embolada	Sem o silêncio de 10 %; durações erradas	`tone(pino, nota, dur*0.9)`; conferir divisores
RGB azul não regula com o buzzer tocando	Conflito `tone` (Timer2) × PWM no pino 3/11	Buzzer em pino sem PWM ou azul no pino 11 → 3 trocar
ESP32 lê distância errada/zero	ECHO sem divisor ou divisor invertido	1 k no ECHO, 2 k no GND, GPIO no meio; medir 3,3 V
ESP32 reinicia (Brownout)	HC-SR04 + buzzer no 3V3 / cabo ruim	Sensor no VIN; cabo curto; porta USB boa
Página não abre	Celular voltou para dados móveis; IP errado	Desligar dados; `192.168.4.1`
`WhatsApp: HTTP -1`	Sem internet na rede; DNS bloqueado	Testar com hotspot; conferir SSID/senha
`WhatsApp: HTTP 203/… sem mensagem`	APIKEY errada ou número sem `+DDI`	Refazer ativação; `+55DDDnúmero`
SMTP `535` na autenticação	Senha normal em vez de senha de app; 2FA desligado	Criar senha de app; remover espaços (o código já faz)
SMTP não conecta (nenhum `220`)	Porta 465 bloqueada pela rede	Hotspot; ou porta 587 com STARTTLS (mais complexo)
Mensagens repetidas sem parar	Cooldown removido/curto; objeto parado no limite	Cooldown ≥ 60 s; histerese (desafio 4.2)

7. Gabarito e comentários dos desafios

Aula 1

- **1.1** `pol = cm / 2.54; mm = cm * 10;` — três regras de três encadeadas.
- **1.2** `if (d < minimo) minimo = d; if (d > maximo) maximo = d;` — inicializar `minimo = 999`, `maximo = 0`.
- **1.3** `if (d < anterior - 1) "<<" else if (d > anterior + 1) ">>" else "=="` — a tolerância é uma histerese simples.
- **1.4** Vetor de 8 + índice `(i + 1) % 8` + soma corrente. Mediana reage mais rápido a degraus; média é mais suave mas atrasa e é puxada por outliers.
- **1.5** `v = 331.3 + 0.606 * T; d = t * (v / 10000.0) / 2;` (v em cm/µs = m/s ÷ 10^4).
- **1.6** Borda: `if (d < 30 && !dentro) { contador++; dentro = true; } if (d >= 30) dentro = false;`.

Aula 2

- **2.1** `v = (d - d_ant) / (t - t_ant)`; sinal indica aproximação (−) ou afastamento (+). Segundo `subplot`.
- **2.2** `import winsound; winsound.Beep(1000, 200)` (Windows) ou `print("\a")`; `datetime.now().strftime("%H:%M:%S")`.
- **2.3** `statistics.mean(list(dists)[-50:])`; cuidado com NaN (filtrar).
- **2.4** Python: `ser.write(b"L")`; Arduino: `if (Serial.available() && Serial.read() == 'L') digitalWrite(13, HIGH);`.
- **2.5** Estados: LONGE → (d < 30 rápido) → PERTO → (d > 60) → LONGE + gesto++. "Rápido" = variação > 20 cm em < 0,5 s.

Aula 3

- **3.1** Dó Dó Ré Dó Fá Mi | Dó Dó Ré Dó Sol Fá | … durações: 8, 8, 4, 4, 4, 2.
- **3.2** `for (r < 3) { 3 × tone(…,150); delay(600); }` + `if (digitalRead(BTN) == LOW) break;`.
- **3.3** `freq = map(pot, 0, 1023, 200, 2000)`; quantização: `idx = map(pot, 0, 1023, 0, 7); tone(pino, escala[idx]);`.
- **3.4** `switch (Serial.read()) { case 'a': tone(8, 262); break; … }` + `noTone` em outra tecla.
- **3.5** Índice da nota + `proximaTroca = millis() + duracao;` no `loop`, se `millis() >= proximaTroca` avança. Mesma estrutura do semáforo com `millis()`.

Aula 4

- **4.1** `if (d < 50) { if (!perto) { perto = true; desde = millis(); } else if (millis() - desde > 2000) alarme = true; } else { perto = false; alarme = false; }`.
- **4.2** Guardar `faixaAtual`; só trocar se `d > LIMITE[faixa] + 2` (sobe) ou `d < LIMITE[faixa-1] - 2` (desce).
- **4.3** A melodia não bloqueante do 3.5 disparada na borda de entrada na faixa 3.

- **4.4** `silencioAte = millis() + 30000` na borda do botão; buzzer só toca se `millis() > silencioAte`.
- **4.5** `LIMITE[2] = map(analogRead(A0), 0, 1023, 10, 100)` — `LIMITE` deixa de ser `const`.
- **4.6** `Serial.print(d); Serial.print(","); Serial.println(faixa);` + `ax.axvspan` ou cor de fundo por faixa no Python.

Aula 5

- **5.1** Vetor circular de 10 `unsigned long`; rota devolve JSON `[…]`; a página lista `(millis()-t)/1000` s atrás.
- **5.2** Rota `/silenciar` → `silencioAte = millis() + 30000`; o buzzer respeita.
- **5.3** Disparar os sensores **alternadamente** (≥ 60 ms entre eles); ecos cruzados são a principal fonte de erro.
- **5.4** `WiFi.begin(ssid, senha)`; IP dinâmico no Serial; celular na mesma rede da escola.
- **5.5** `requests.get("http://192.168.4.1/dados").json()["cm"]` a cada 0,3 s no lugar de `fonte.ler()` — troca-se só a classe da fonte (abstração da Aula 2).

Aula 6

- **6.1** `configTime(-3*3600, 0, "pool.ntp.org"); struct tm t; getLocalTime(&t); strftime(...)`.
- **6.2** Contador diário + verificação `t.tm_hour == 18 && !enviadoHoje`.
- **6.3** `xTaskCreatePinnedToCore(tarefaEnvio, "envio", 8192, NULL, 1, NULL, 0)` com uma fila (`QueueHandle_t`) recebendo os textos.
- **6.4** `bool z = enviarWhatsApp(); bool e = enviarEmail(); if (z && e) verde3x(); else vermelho3x();` — discutir os 4 casos.
- **6.5** Telegram: `https://api.telegram.org/bot<TOKEN>/sendMessage?chat_id=<ID>&text=…` (mesmo `HTTPClient`). ntfy: `http.POST(texto)` em `https://ntfy.sh/<topico>`. Telegram costuma ser o mais robusto e permite grupos.
- **6.6** `WiFi.mode(WIFI_AP_STA); WiFi.softAP(...); WiFi.begin(...);` e manter o `WebServer`.

8. Avaliação

Rubrica do projeto final

Critério	Iniciante (1)	Em desenvolvimento (2)	Proficiente (3)
Medição e física	Lê a distância, não explica a fórmula	Explica a regra de três e um fator de erro	Explica fórmula, filtros e fatores (temperatura, ângulo); mede com erro < 5 %
Programação e lógica	Reproduz sketches	Explica `millis`, tabelas e bordas; resolveu 1 desafio por aula	Modifica com propósito; resolveu desafios avançados (histerese, tarefa paralela)
Dados e Python	Roda o script	Explica `split`/`float`/NaN; gera CSV	Cria novo gráfico/estatística; comunica via serial nos dois

Critério	Iniciante (1)	Em desenvolvimento (2)	Proficiente (3)
			sentidos
Integração e rede	Página abre	Página com gráfico e alarme funcionando	Local + internet; explica HTTP/JSON/SMTP; cooldown e credenciais corretos
Depuração e comunicação	Depende do professor	Usa Serial/`repr`/tabela de erros	Diagnostica sozinho; apresentação clara com demonstração

Formativa

- Tickets de saída (um por aula, nos planos).
- Ficha de medições da Aula 1 (erro absoluto e relativo).
- CSV + planilha da Aula 2.
- Tabela de estados escrita antes do código na Aula 4.
- Autoavaliação dos 8 objetivos do §1.

9. Diferenciação

- Desafios em escada em todas as aulas; nenhum grupo fica parado.
- **Avançados:** 1.4/1.5 (filtros e correção), 2.5 (gestos), 3.5 (melodia não bloqueante), 4.2 (histerese), 5.5 (Python via Wi-Fi), 6.3 (FreeRTOS).
- **Dificuldade:** começar sempre pelo "Experimente"; sketches com `// <<< MUDE AQUI`; usar o simulador Python quando a montagem atrasar.
- **Sem internet na escola:** Aula 6 com hotspot, ou substituir por Telegram via hotspot, ou fazer a Aula 6 como "leitura guiada do protocolo" + demonstração do professor.
- **Sem ESP32:** Aulas 1–4 são completas; a Aula 5 pode ser feita com o Python servindo a página (Flask) a partir da serial — variante para turmas de informática.

10. Segurança

- Divisor de tensão obrigatório no ESP32.
- Credenciais em arquivo separado, distribuídas na hora, revogadas após a aula (senha de app pode ser apagada; CallMeBot pode ser desativado).
- Combinar com a turma o uso responsável do envio de mensagens (cooldown; nada de mandar para terceiros — a API nem permite).
- Buzzer alto por muito tempo incomoda: pausas e volume (resistor em série de 100 Ω reduz).

11. Referências

- HC-SR04 — datasheet (Elec Freaks / Cytron): faixa 2–400 cm, 15°, pulso ≥ 10 µs, ciclo ≥ 60 ms.
- Arduino — `pulseIn`, `tone`, Serial: https://www.arduino.cc/reference/pt/

- pyserial: https://pyserial.readthedocs.io/ • matplotlib animation: https://matplotlib.org/stable/api/animation_api.html
- Robson Couto — arduino-songs: https://github.com/robsoncouto/arduino-songs
- CallMeBot WhatsApp API: https://www.callmebot.com/blog/free-api-whatsapp-messages/
- Google — senhas de app: https://support.google.com/accounts/answer/185833 • RFC 5321 (SMTP)
- ESP32 Arduino Core (WiFi, WebServer, WiFiClientSecure, LEDC, FreeRTOS): https://docs.espressif.com/projects/arduino-esp32/
- Sketches e scripts deste curso: `aula-distancia/codigo/` e `aula-distancia/python/`.

Anexo A — Esquemas

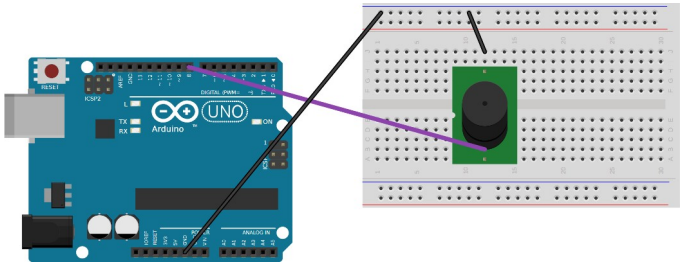


Figura 5 — Aula 3 — buzzer.

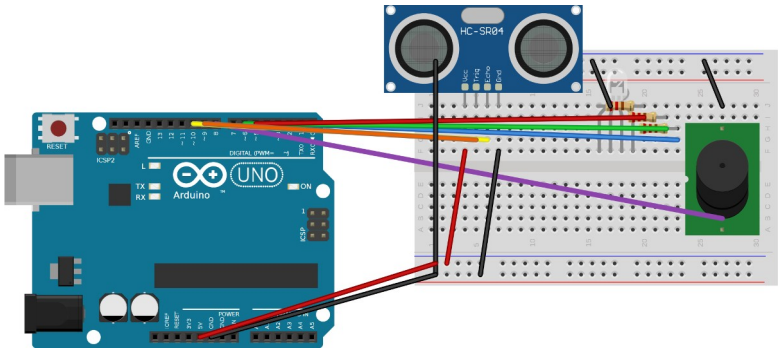


Figura 6 — Aula 4 — alarme de ré.

Anexo B — Sketch final de referência

05_esp32_alerta_internet.ino

```
/*
  05 - Alerta pela INTERNET: WhatsApp (CallMeBot) e e-mail (SMTP do Gmail)
  Placa: ESP32 Dev Module (core 3.x)   Sem bibliotecas extras.
  Ligacoes: iguais ao sketch 04

  O e-mail e enviado "na mao", conversando com o servidor SMTP do Gmail em TEXTO PUTO:
  EHLO, AUTH LOGIN, MAIL FROM, RCPT TO, DATA, QUIT. Abra o Monitor Serial e veja o dialogo -
  e o mesmo protocolo de 1982 que todo programa de e-mail usa por baixo dos panos.
  (Alternativa com biblioteca: "ESP Mail Client", de Mobizt - precisa da particao "Huge APP".)

  Diferenca em relacao ao sketch 04: agora o ESP32 ENTRA na rede Wi-Fi da escola (que tem internet)
  em vez de criar a propria. Quando um objeto passa do limite, ele:
     1) manda uma mensagem de WhatsApp pela API gratuita CallMeBot
     2) manda um e-mail pelo servidor SMTP do Gmail
  Ha um "tempo de silencio" (COOLDOWN) para nao mandar 50 mensagens num unico evento.

  --- Preparacao (uma vez) ---
  WhatsApp: salve o contato +34 644 87 21 57, mande "I allow callmebot to send me messages" e
            anote a APIKEY que vem na resposta (uso pessoal; so manda para o SEU numero).
  E-mail:   na conta Google, ative a verificacao em 2 etapas e crie uma "Senha de app" (16 letras).
            Use essa senha aqui, NUNCA a senha normal.
*/
#include <WiFi.h>
#include <WiFiClientSecure.h>
#include <HTTPClient.h>
#include <base64.h>

// ---------- preencha ----------
const char* WIFI_SSID  = "NOME-DA-REDE";
const char* WIFI_SENHA = "SENHA-DA-REDE";
const char* ZAP_FONE   = "+5581999999999";     // seu numero com DDI+DDD
const char* ZAP_APIKEY = "123456";             // apikey do CallMeBot
const char* EMAIL_DE   = "seu.email@gmail.com";
const char* EMAIL_SENHA_APP = "abcd efgh ijkl mnop";
const char* EMAIL_PARA = "destino@exemplo.com";
```

```
// ---------- hardware ----------
const int TRIG = 5, ECHO = 18, BUZZER = 19;
const int PINO_R = 25, PINO_G = 26, PINO_B = 27;
const int LIMITE_CM = 40;
const unsigned long COOLDOWN_MS = 60000;       // no maximo 1 alerta por minuto

unsigned long ultimoAlerta = 0;
bool emAlarme = false;
int totalAlarmes = 0;

float lerDistancia() {
  digitalWrite(TRIG, LOW);  delayMicroseconds(2);
  digitalWrite(TRIG, HIGH); delayMicroseconds(10);
  digitalWrite(TRIG, LOW);
  unsigned long t = pulseIn(ECHO, HIGH, 30000);
  return (t == 0) ? -1 : t * 0.0343 / 2.0;
}
void cor(int r, int g, int b) { ledcWrite(PINO_R, r); ledcWrite(PINO_G, g); ledcWrite(PINO_B, b); }

// ---------- WhatsApp ----------
String urlEncode(String s) {                    // espaco -> %20 etc.
  String out = "";
  const char* hex = "0123456789ABCDEF";
  for (unsigned int i = 0; i < s.length(); i++) {
    char c = s[i];
    if (isalnum(c) || c == '-' || c == '_' || c == '.' || c == '~') out += c;
    else { out += '%'; out += hex[(c >> 4) & 0xF]; out += hex[c & 0xF]; }
  }
  return out;
}
bool enviarWhatsApp(String texto) {
  WiFiClientSecure cliente;
  cliente.setInsecure();                        // nao valida o certificado (ok para a aula)
  HTTPClient http;
  String url = "https://api.callmebot.com/whatsapp.php?phone=" + String(ZAP_FONE) +
               "&text=" + urlEncode(texto) + "&apikey=" + String(ZAP_APIKEY);
  http.begin(cliente, url);
  int codigo = http.GET();
  http.end();
  Serial.printf("WhatsApp: HTTP %d\n", codigo);
  return codigo == 200;
}

// ---------- E-mail (SMTP "na mao") ----------
// manda uma linha ao servidor e espera a resposta; o SMTP responde com um codigo de 3 digitos
bool smtpFala(WiFiClientSecure& c, String linha, int esperado) {
  if (linha.length()) { c.println(linha); Serial.println("> " + (linha.startsWith("AUTH") ||
linha.length() == 24 ? "(credencial)" : linha)); }
  unsigned long t0 = millis();
  while (!c.available() && millis() - t0 < 8000) delay(10);
  String resp = c.readStringUntil('\n');
  Serial.println("< " + resp);
  return resp.startsWith(String(esperado));
}

bool enviarEmail(String assunto, String texto) {
  WiFiClientSecure c;
  c.setInsecure();
  if (!c.connect("smtp.gmail.com", 465)) { Serial.println("SMTP: nao conectou"); return false; }
  String senha = EMAIL_SENHA_APP; senha.replace(" ", "");           // a senha de app vem com espacos
  bool ok = smtpFala(c, "", 220)
         && smtpFala(c, "EHLO esp32", 250)
         && smtpFala(c, "AUTH LOGIN", 334)
         && smtpFala(c, base64::encode(EMAIL_DE), 334)
         && smtpFala(c, base64::encode(senha), 235)
         && smtpFala(c, "MAIL FROM:<" + String(EMAIL_DE) + ">", 250)
         && smtpFala(c, "RCPT TO:<" + String(EMAIL_PARA) + ">", 250)
         && smtpFala(c, "DATA", 354);
  if (ok) {
    c.println("From: Alarme ESP32 <" + String(EMAIL_DE) + ">");
    c.println("To: <" + String(EMAIL_PARA) + ">");
    c.println("Subject: " + assunto);
    c.println("Content-Type: text/plain; charset=utf-8");
```

```
    c.println();
    c.println(texto);
    ok = smtpFala(c, ".", 250);                                    // linha so com "." = fim da mensagem
  }
  smtpFala(c, "QUIT", 221);
  c.stop();
  Serial.println(ok ? "E-mail enviado!" : "E-mail: falhou (veja o dialogo acima)");
  return ok;
}

void setup() {
  Serial.begin(115200);
  pinMode(TRIG, OUTPUT); pinMode(ECHO, INPUT); pinMode(BUZZER, OUTPUT);
  ledcAttach(PINO_R, 5000, 8); ledcAttach(PINO_G, 5000, 8); ledcAttach(PINO_B, 5000, 8);

  WiFi.begin(WIFI_SSID, WIFI_SENHA);
  Serial.print("Conectando ao Wi-Fi");
  while (WiFi.status() != WL_CONNECTED) { delay(500); Serial.print("."); }
  Serial.print("\nConectado. IP: "); Serial.println(WiFi.localIP());
  cor(0, 255, 0);
}

void loop() {
  float d = lerDistancia();
  bool agora = (d > 0 && d < LIMITE_CM);

  if (agora && !emAlarme) {                                    // BORDA: acabou de entrar em alarme
    totalAlarmes++;
    Serial.printf("ALARME #%d: %.0f cm\n", totalAlarmes, d);
    cor(255, 0, 0); tone(BUZZER, 2200, 300);
    if (millis() - ultimoAlerta > COOLDOWN_MS || ultimoAlerta == 0) {
      ultimoAlerta = millis();
      String texto = "ALERTA: objeto detectado a " + String(d, 0) + " cm (alarme #" + String(totalAlarmes)
+ ")";
      cor(0, 0, 255);                                           // azul = enviando
      enviarWhatsApp(texto);                                    // ~2 s
      enviarEmail("Alarme ESP32 disparou", texto);              // ~5 s
      cor(255, 0, 0);
    } else Serial.println("(dentro do cooldown, nao envia de novo)");
  }
  if (!agora && emAlarme) cor(0, 255, 0);
  emAlarme = agora;
  delay(100);
}
```